

Proyecto 2 – 2026 – Jerarquía de memoria en MIPS

Polo Gracia, Gonzalo

Serrano Fernández, Alberto

lunes, 11 de mayo de 2026

Índice

Resumen.....	3
Grafo de estados de la UC.....	3
Desglose de direcciones.....	10
Análisis de latencias.....	10
Coste medio por acceso.....	11
Pruebas.....	13
Prueba de integración en RAM.....	13
Test 1. Integración larga: Test_lecturas_P2.....	13
Test 2. write-around en fallo de escritura.....	14
Test 3. Reemplazo sucio y copy-back.....	15
Test 4. Accesos a MD Scratch.....	17
Test 5. Registros de E/S.....	17
Test 6. Escritura ilegal en registro interno.....	18
Test 7. Dirección fuera de rango / ausencia de DevSel.....	19
Test 8. Acceso desalineado.....	19
Test 9. UNDEF.....	20
Tabla de cobertura por estado.....	21
Análisis de speedup.....	21
Cuantificación de horas dedicadas.....	23
Conclusiones y autoevaluación.....	23

Resumen

Se ha implementado la unidad de control de una jerarquía de memoria para un MIPS segmentado de 5 etapas. La solución integra una caché asociativa de dos vías con reemplazo FIFO, política copy-back y write-around en fallo de escritura, además de soporte para MD Scratch, registros internos y gestión de errores de memoria. La UC controla arbitraje, envío de dirección, transferencias por palabra o por bloque, reemplazos sucios y borrado del estado de error al leer x"01000000". Se han medido las latencias pedidas por el guion y se ha formulado el coste medio por acceso usando dichas medidas.

Grafo de estados de la UC

La unidad de control se implementa como una máquina de estados finitos con una FSM principal para el protocolo de acceso a memoria y una FSM secundaria para la señal Mem_ERROR. La FSM principal distingue tres familias de accesos:

- accesos resueltos localmente: read hit, write hit, lectura de registro interno y detección inmediata de error
- accesos de una sola palabra: MD Scratch, escritura con write-around y accesos directos no cacheables
- accesos por bloque: read miss limpio y read miss con reemplazo sucio y copy-back

La figura siguiente recoge el grafo de estados implementado:



La tabla siguiente resume, de forma compacta, las señales de estado, las condiciones de transición y las activaciones de tipo Mealy relevantes.

TRANSICIÓN	ESTADO	SEÑALES DE ESTADO	CONDICIÓN	SIGUIENTE ESTADO	SEÑALES QUE SE ACTIVAN
T1	Inicio	count_reset, ready	RE = '0' AND WE = '0'	Inicio	-
T2	Inicio	count_reset, ready	((RE='1') OR (WE='1')) AND unaligned='1'	Inicio	load_addr_e rror, next_error_s tate = memory_err or
T3	Inicio	count_reset, ready	RE = '1' AND internal_add r = '1'	Inicio	mux_output = "10", next_error_s tate = No_error
T4	Inicio	count_reset, ready	WE = '1' AND internal_add r = '1'	Inicio	load_addr_e rror, next_error_s tate = memory_err or
T5	Inicio	count_reset, ready	RE = '1' AND hit = '1'	Inicio	inc_r, mux_output = "00"
T6	Inicio	count_reset, ready	WE = '1' AND hit = '1'	Inicio	inc_w, Update_dirty , MC_WE0 = hit0, MC_WE1 = hit1
T7	Inicio	count_reset, ready	(RE = '1' AND hit = '0') AND addr_non_ca cheable = '1'	single_word _transfer_ad dr	ready = '0'

TRANSICIÓN	ESTADO	SEÑALES DE ESTADO	CONDICIÓN	SIGUIENTE ESTADO	SEÑALES QUE SE ACTIVAN
T8	Inicio	count_reset, ready	RE = '1' AND hit = '0' AND addr_non_cacheable = '0' AND dirty_bit_rpl = '1'	Send_ADDR_CB	ready = '0', inc_r, inc_m
T9	Inicio	count_reset, ready	RE = '1' AND hit = '0' AND addr_non_cacheable = '0' AND dirty_bit_rpl = '0'	block_transfer_addr	ready = '0', inc_r, inc_m
T10	Inicio	count_reset, ready	(WE = '1' AND hit = '0') AND addr_non_cacheable = '1'	single_word_transfer_addr	ready = '0'
T11	Inicio	count_reset, ready	(WE = '1' AND hit = '0') AND addr_non_cacheable = '0'	single_word_transfer_addr	ready = '0', inc_m
T12	single_word_transfer_addr	count_reset, Bus_req, ready = '0'	Bus_grant = '1'	Send_Addr	-
T13	Send_Addr	Frame, MC_send_addr_ctrl, block_addr = '0', MC_bus_Read = RE, MC_bus_Write = WE	Bus_DevSel = '1'	single_word_transfer_data	-

TRANSICIÓN	ESTADO	SEÑALES DE ESTADO	CONDICIÓN	SIGUIENTE ESTADO	SEÑALES QUE SE ACTIVAN
T14	Send_Addr	Frame, MC_send_addr_ctrl, block_addr = '0', MC_bus_Read = RE, MC_bus_Write = WE	Bus_DevSel = '0'	fallo	-
T15	single_word_transfer_data	ready = '0', Frame, last_word	WE = '1'	single_word_transfer_data	MC_send_data
T16	single_word_transfer_data	ready = '0', Frame, last_word	WE = '0'	single_word_transfer_data	mux_output = "01"
T17	single_word_transfer_data	ready = '0', Frame, last_word	bus_TRDY = '1'	Inicio	ready
T18	block_transfer_addr	count_reset, ready = '0', Bus_req	Bus_grant = '1'	read_block	-
T19	read_block	Frame, MC_send_addr_ctrl, MC_bus_Read, block_addr	Bus_DevSel = '1'	block_transfer_data	-
T20	read_block	Frame, MC_send_addr_ctrl, MC_bus_Read, block_addr	Bus_DevSel = '0'	fallo	-
T21	block_transfer_data	ready = '0', Frame,	bus_TRDY = '1' AND	bajar_Frame	MC_tags_W E

TRANSICIÓN	ESTADO	SEÑALES DE ESTADO	CONDICIÓN	SIGUIENTE ESTADO	SEÑALES QUE SE ACTIVAN
		mux_origen, MC_WE0 = bus_TRDY AND (NOT via_2_rpl), MC_WE1 = bus_TRDY AND via_2_rpl, count_enabl e = bus_TRDY, last_word = last_word_bl ock	last_word_bl ock = '1'		
T22	block_transf er_data	ready = '0', Frame, mux_origen, MC_WE0 = bus_TRDY AND (NOT via_2_rpl), MC_WE1 = bus_TRDY AND via_2_rpl, count_enabl e = bus_TRDY, last_word = last_word_bl ock	bus_TRDY = '1' AND last_word_bl ock = '0'	block_transf er_data	-
T23	Send_ADDR_ CB	count_reset, ready = '0', Bus_req	Bus_grant = '1'	write_dirty_ block	-
T24	write_dirty_ block	Frame, MC_send_ad dr_ctrl,	Bus_DevSel = '1'	CopyBack	-

TRANSICIÓN	ESTADO	SEÑALES DE ESTADO	CONDICIÓN	SIGUIENTE ESTADO	SEÑALES QUE SE ACTIVAN
		MC_bus_wri te, send_dirty, inc_cb			
T25	write_dirty_ block	Frame, MC_send_ad dr_ctrl, MC_bus_wri te, send_dirty, inc_cb	Bus_DevSel = '0'	fallo	-
T26	CopyBack	ready = '0', Frame, mux_origen, send_dirty, MC_send_da ta, last_word = last_word_bl ock, count_enabl e = bus_TRDY	bus_TRDY = '1' AND last_word_bl ock = '1'	block_transf er_addr	Block_copie d_back, Update_dirty
T27	CopyBack	ready = '0', Frame, mux_origen, send_dirty, MC_send_da ta, last_word = last_word_bl ock, count_enabl e = bus_TRDY	bus_TRDY = '1' AND last_word_bl ock = '0'	CopyBack	-
T28	fallo	count_reset, ready,	-	Inicio	-

TRANSICIÓN	ESTADO	SEÑALES DE ESTADO	CONDICIÓN	SIGUIENTE ESTADO	SEÑALES QUE SE ACTIVAN
		next_error_state = memory_error, load_address_error			
T29	bajar_Frame	count_reset, ready, mux_output = "00"	-	Inicio	-

Observaciones de diseño:

- El inicio de una transferencia de bus se retrasa al ciclo siguiente de Bus_grant. Esta separación evita el lazo combinacional entre gestor y árbitro.
- Los read hit y write hit no insertan esperas en el MIPS; el acceso se resuelve en el mismo estado Inicio.
- El write miss cacheable no asigna bloque en MC. La UC fuerza write-around y reenvía la palabra a MD.
- El reemplazo sucio consta de dos ráfagas consecutivas: primero copy-back del bloque víctima y después lectura del nuevo bloque.
- La FSM secundaria mantiene Mem_ERROR = '1' hasta que se lee x"01000000".

Desglose de direcciones

La caché es asociativa de dos vías y almacena 128 bytes, es decir, 32 palabras de 32 bits. Cada vía contiene 4 conjuntos y cada conjunto contiene un bloque de 4 palabras. El desglose de la dirección se hace como sigue:

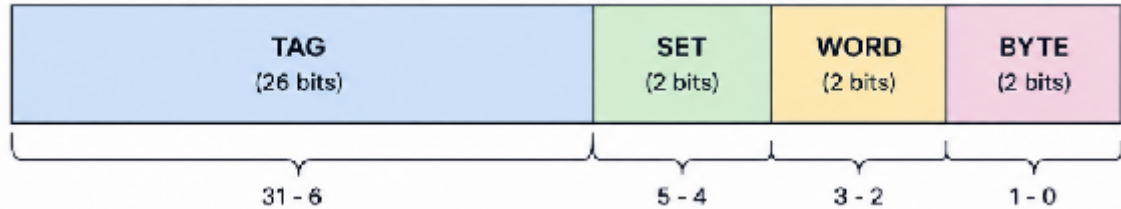
- byte = ADDR(1 downto 0)
- word = ADDR(3 downto 2)
- set = ADDR(5 downto 4)
- tag = ADDR(31 downto 6)

Interpretación:

- Los bits 1..0 seleccionan el byte dentro de la palabra.
- Los bits 3..2 seleccionan la palabra dentro del bloque.
- Los bits 5..4 seleccionan el conjunto.

- Los bits 31..6 forman la etiqueta que se compara con las etiquetas almacenadas en ambas vías del conjunto seleccionado.

La figura siguiente resume este desglose:



Análisis de latencias

Las latencias se han medido por simulación, excluyendo arbitraje, tal y como pide el guion. La cuenta empieza en el primer ciclo en que el gestor presenta la fase de dirección al módulo destino y termina en el ciclo en que la transferencia queda completada.

Resultados medidos:

Magnitud	Valor
CrB(MD)	17 ciclos
CwW(MD)	6 ciclos
CrW(MDscratch)	2 ciclos
CwW(MDscratch)	1 ciclo
CrW(IO_REG)	1 ciclo
CwW(IO_REG)	1 ciclo
CrW(MD)	6 ciclos

Notas:

- CrB(MD) corresponde a la lectura de un bloque de 4 palabras desde MD.
- CrW(MD) no es una magnitud pedida explícitamente por el PDF, pero sí es necesaria para comparar contra un sistema equivalente con solo MD.
- En la ruta integrada se ha observado además:
 - read hit = 0 ciclos de espera
 - write hit = 0 ciclos de espera
 - read miss limpio entre 16 y 20 ciclos observados, según el instante de arbitraje y la alineación del acceso dentro de la secuencia integrada

- read miss con reemplazo sucio: 33 ciclos observados en la secuencia de validación dirigida
- write-around: 9 ciclos observados en la secuencia de validación dirigida

La diferencia entre read miss limpio y read miss con reemplazo sucio permite inferir que el coste del copy-back de un bloque completo es también de 17 ciclos. En consecuencia:

- $C_{cbB}(MD) = 17$ ciclos

Coste medio por acceso

Sea $A = 1.5$ el coste medio de arbitraje pedido por el guión.

Se usan las siguientes categorías de acceso:

- P_{rh} : probabilidad de read hit
- P_{wh} : probabilidad de write hit
- P_{rm_c} : probabilidad de read miss limpio a MD
- P_{rm_d} : probabilidad de read miss con reemplazo sucio a MD
- P_{wm} : probabilidad de write miss cacheable a MD
- P_{rs} : probabilidad de lectura a MD Scratch
- P_{ws} : probabilidad de escritura a MD Scratch
- P_{ri} : probabilidad de lectura de registro interno
- P_{wi} : probabilidad de escritura de registro interno

Con:

- $P_{rh} + P_{wh} + P_{rm_c} + P_{rm_d} + P_{wm} + P_{rs} + P_{ws} + P_{ri} + P_{wi} = 1$

Los costes por tipo de evento quedan:

- $C_{rh} = 0$
- $C_{wh} = 0$
- $C_{rm_c} = A + C_{rB}(MD) = 1.5 + 17 = 18.5$
- $C_{rm_d} = 2A + C_{cbB}(MD) + C_{rB}(MD) = 3 + 17 + 17 = 37$
- $C_{wm} = A + C_{wW}(MD) = 1.5 + 6 = 7.5$
- $C_{rs} = A + C_{rW}(MD_{scratch}) = 1.5 + 2 = 3.5$
- $C_{ws} = A + C_{wW}(MD_{scratch}) = 1.5 + 1 = 2.5$
- $C_{ri} = C_{rW}(IO_REG) = 1$
- $C_{wi} = C_{wW}(IO_REG) = 1$

Por tanto, el coste medio por acceso a memoria es:

$C_{medio} =$
 $P_{rh} * 0$

+ P_wh * 0
+ P_rm_c * 18.5
+ P_rm_d * 37
+ P_wm * 7.5
+ P_rs * 3.5
+ P_ws * 2.5
+ P_ri * 1
+ P_wi * 1

Interpretación:

- Los hit no insertan ciclos de espera adicionales en el procesador.
- El peor caso es el read miss con reemplazo sucio, ya que concatena dos arbitrajes y dos transferencias por bloque.
- Los accesos a MD Scratch son claramente más baratos que un fallo cacheable a MD.
- Las escrituras con write-around tienen un coste próximo al de un acceso directo por palabra a MD.

Pruebas

Prueba de integración en RAM

Test 1. Integración larga: Test_lecturas_P2

Objetivo:

- comprobar la evolución de los contadores durante un flujo largo de lecturas
- comprobar el paso final a la rutina de abort

Resultado esperado:

- para R5 = 1..31 se cumple:
 - m = R5 + 1
 - w = 0
 - r = 2*R5 + 2
 - cb = 0
- a partir de R5 = 32 el programa entra en la zona de abort y la pauta deja de ser la del bucle normal
- debe aparecer el marcador 0x00000AB0 y, después, la dirección de error

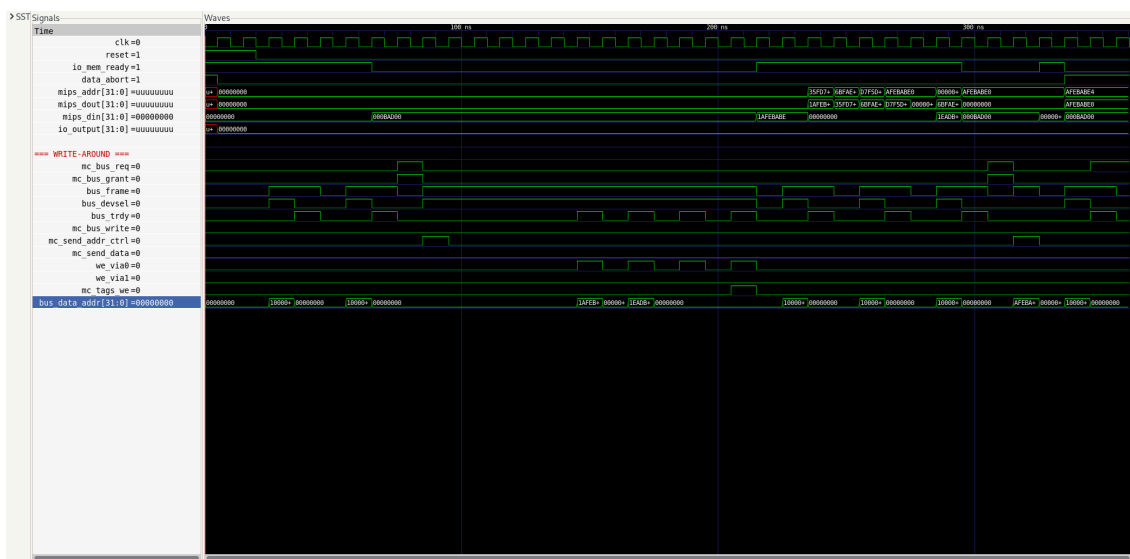
- demostrar que un write miss cacheable no asigna bloque en MC
- demostrar que la palabra se escribe directamente en MD

Acceso relevante:

- sw @0x000000C0

Clasificación:

- miss
- set = 0
- no queda línea cargada en caché tras la escritura



Snippet:

```
lw R1, 0(R0)      ; carga valor base desde MD

sw R1, 192(R0)    ; sw @0x000000C0, write miss cacheable

lw R2, 192(R0)    ; primera lectura posterior, debe ser miss

sw R2, 0x7004(R0) ; visible en IO_output

lw R2, 192(R0)    ; segunda lectura, ya debe ser hit

sw R2, 0x7004(R0)
```

Test 3. Reemplazo sucio y copy-back

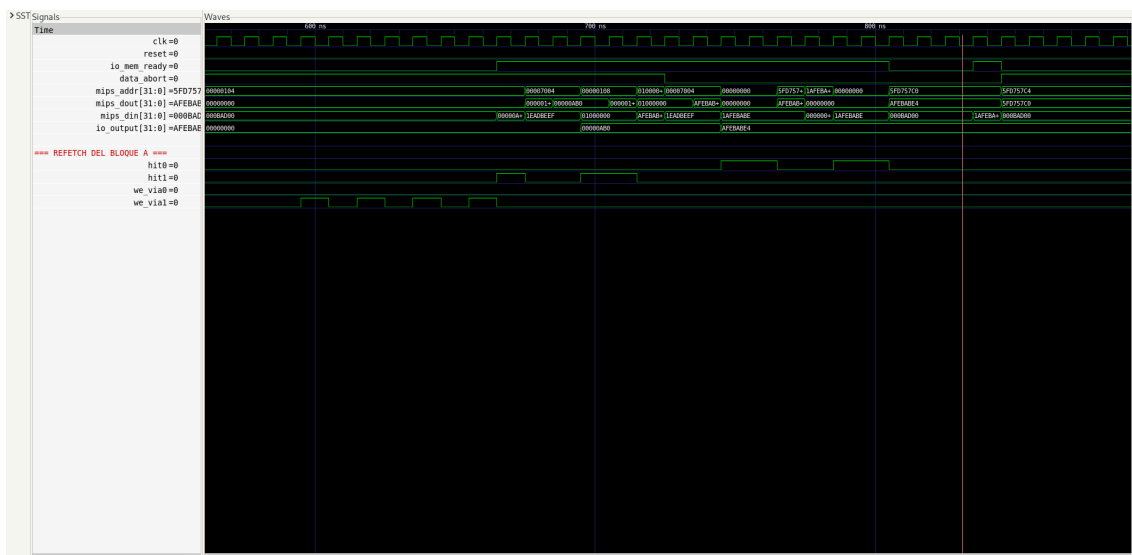
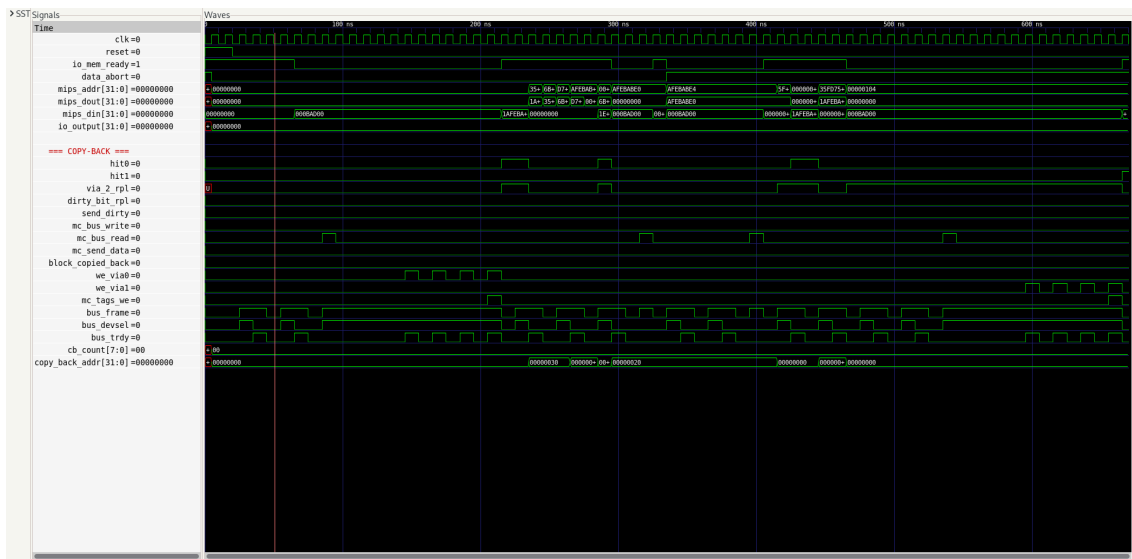
Objetivo:

- demostrar el reemplazo FIFO

- demostrar la escritura previa del bloque sucio en MD
- demostrar la recarga posterior del bloque nuevo

Secuencia relevante:

1. lw @0x00000000 -> miss, set = 0, entra en vía 0
2. sw @0x00000000 -> hit, set = 0, vía 0 queda sucia
3. lw @0x00000040 -> miss, set = 0, entra en vía 1
4. lw @0x00000080 -> miss, set = 0, reemplazo FIFO de vía 0 con copy-back



Snippet:

```
lw R1, 0(R0)      ; A: miss, entra en via 0

lw R2, 4(R0)      ; hit en el mismo bloque

sw R2, 0(R0)      ; write hit, bloque A queda sucio

lw R3, 64(R0)     ; B: miss, entra en via 1

lw R4, 128(R0)    ; C: miss, reemplazo FIFO con copy-back

lw R2, 0(R0)      ; refetch de A, debe recuperar el valor modificado

sw R2, 0x7004(R0)
```

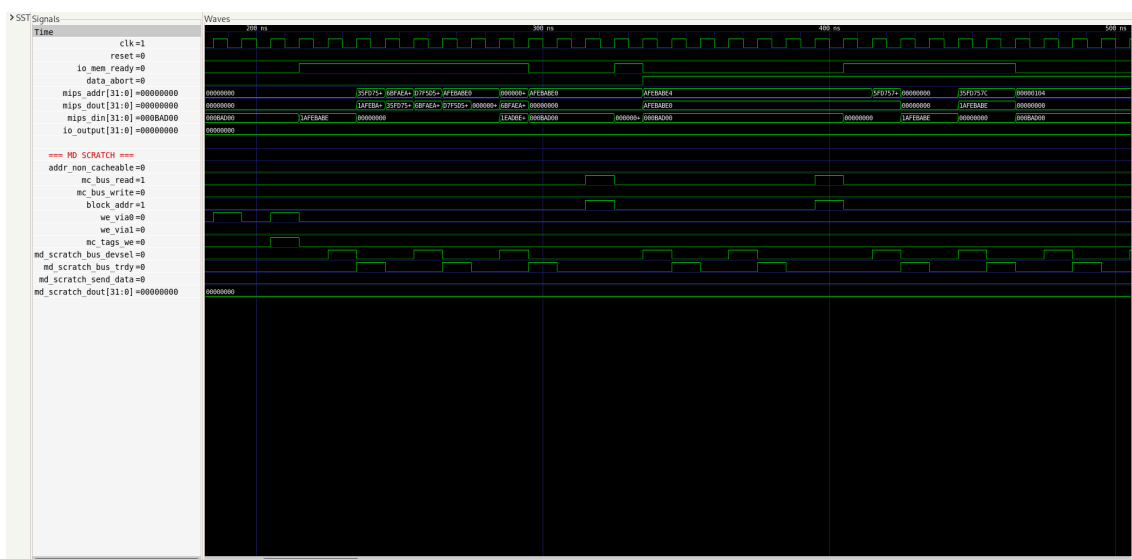
Test 4. Accesos a MD Scratch

Objetivo:

- demostrar que MD Scratch se accede por palabra
- demostrar que estos accesos no generan llenado de caché

Accesos relevantes:

- sw @0x10000004
- lw @0x10000004



Snippet:

```
lw R1, 0(R0)      ; valor a escribir
```



```
lw R3, 256(R0)    ; carga 0x10000000 desde RAM de datos
sw R1, 4(R3)      ; sw @0x10000004 hacia MD Scratch
lw R2, 4(R3)      ; lw @0x10000004 desde MD Scratch
sw R2, 0x7004(R0)
```

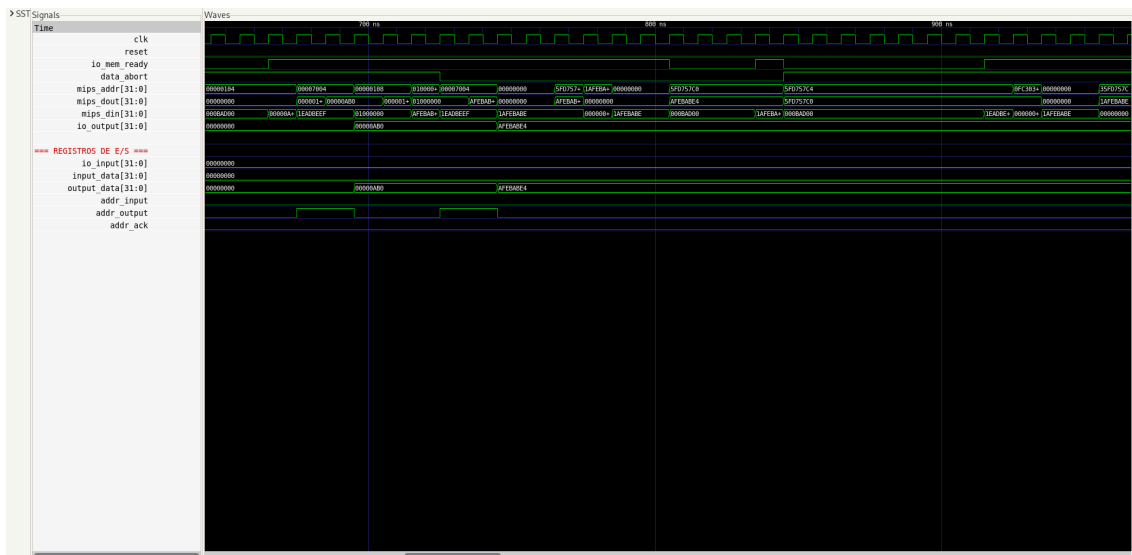
Test 5. Registros de E/S

Objetivo:

- demostrar lectura y escritura correctas de registros de E/S

Accesos relevantes:

- lectura de 0x00007000
- escritura de 0x00007004



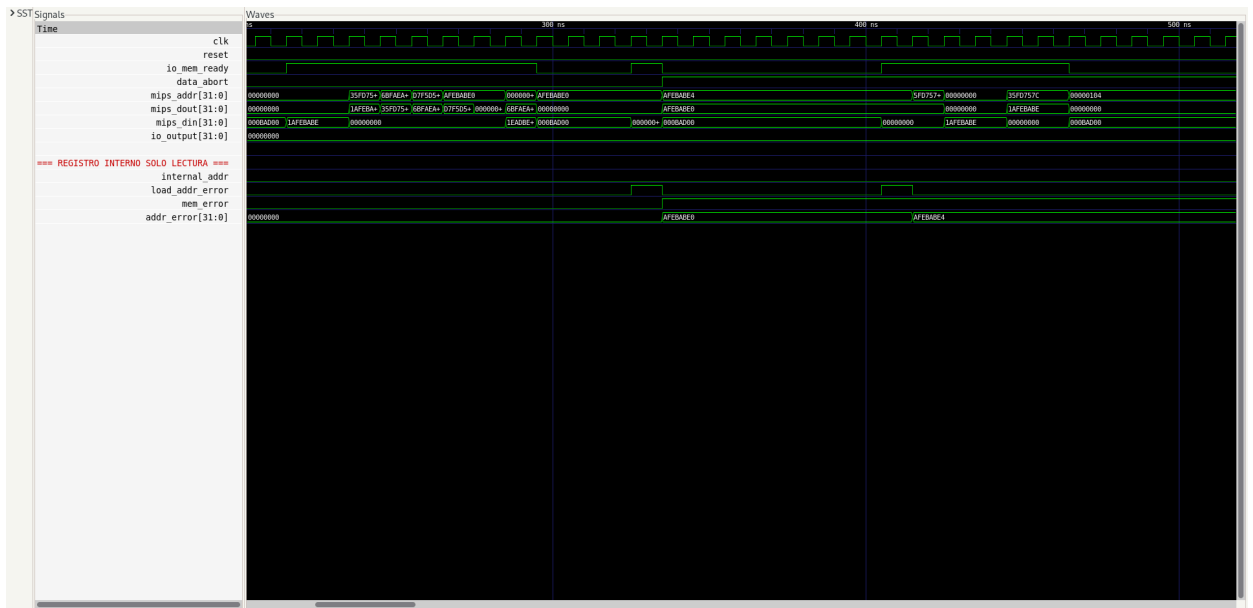
Snippet:

```
lw R1, 0x7000(R0) ; lectura de IO_input
sw R1, 0x7004(R0) ; escritura en IO_output
lw R2, 0(R0)      ; carga de un valor auxiliar
sw R2, 0x7008(R0) ; escritura en INT_ACK
```

Test 6. Escritura ilegal en registro interno

Objetivo:

- demostrar que la escritura en Addr_Error_Reg genera error



Snippet:

lw R1, 264(R0) ; carga 0x01000000 desde RAM de datos

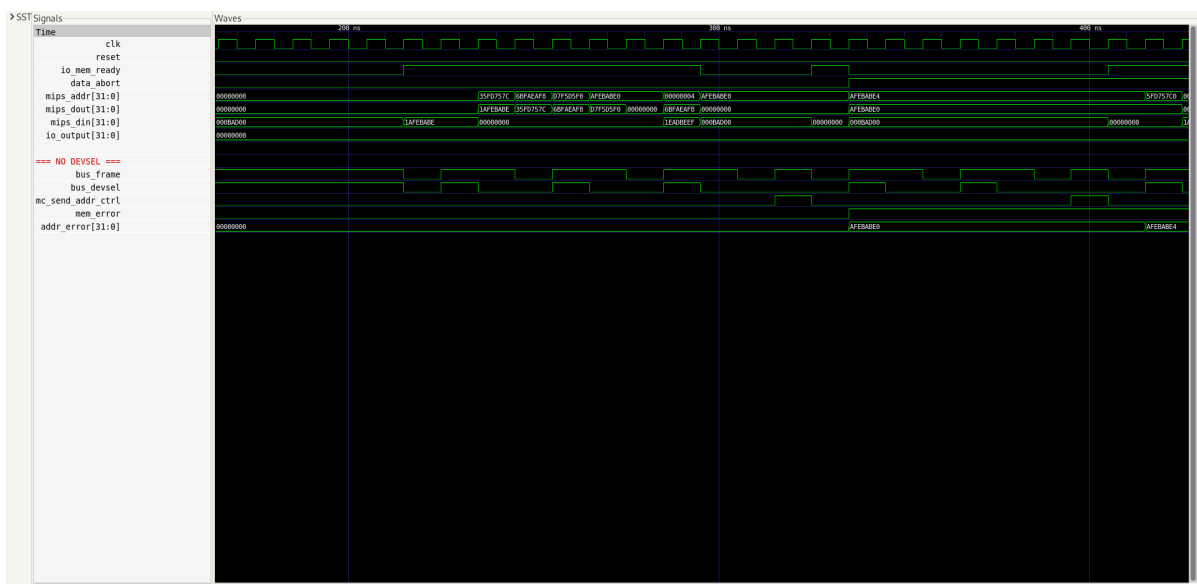
lw R2, 0(R0) ; valor a escribir

sw R2, 0(R1) ; intento de escritura en Addr_Error_Reg

Test 7. Dirección fuera de rango / ausencia de DevSel

Objetivo:

- demostrar el camino de fallo cuando ninguna memoria responde a la dirección de bus



Snippet:

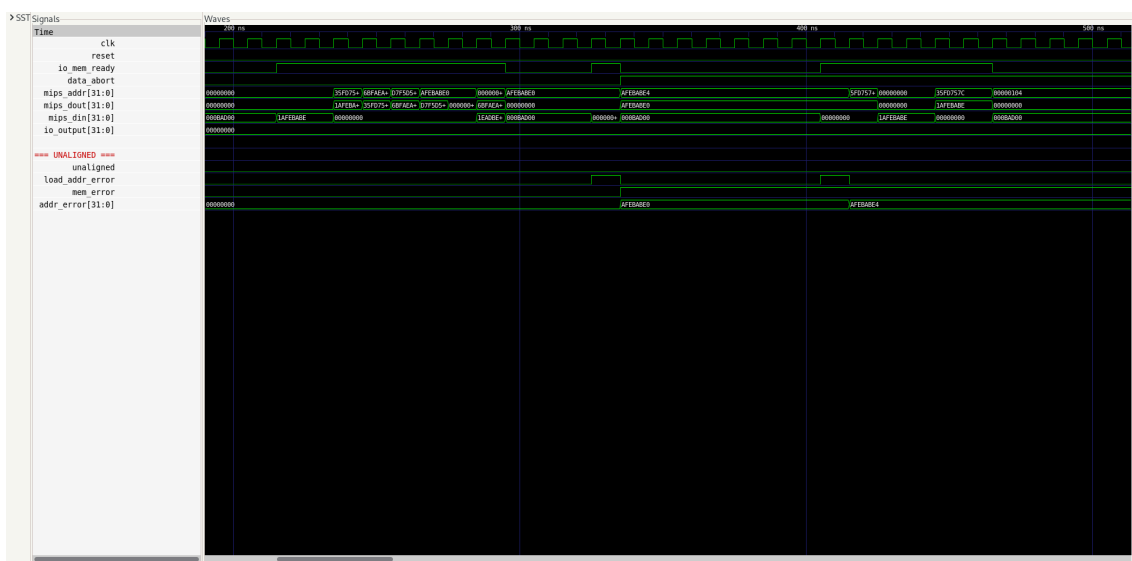
lw R1, 272(R0) ; carga 0x20000000 desde RAM de datos

lw R2, 0(R1) ; acceso fuera de rango, nadie responde con DevSel

Test 8. Acceso desalineado

Objetivo:

- demostrar detección inmediata de acceso desalineado
- demostrar que el registro de error guarda la dirección exacta del byte erróneo



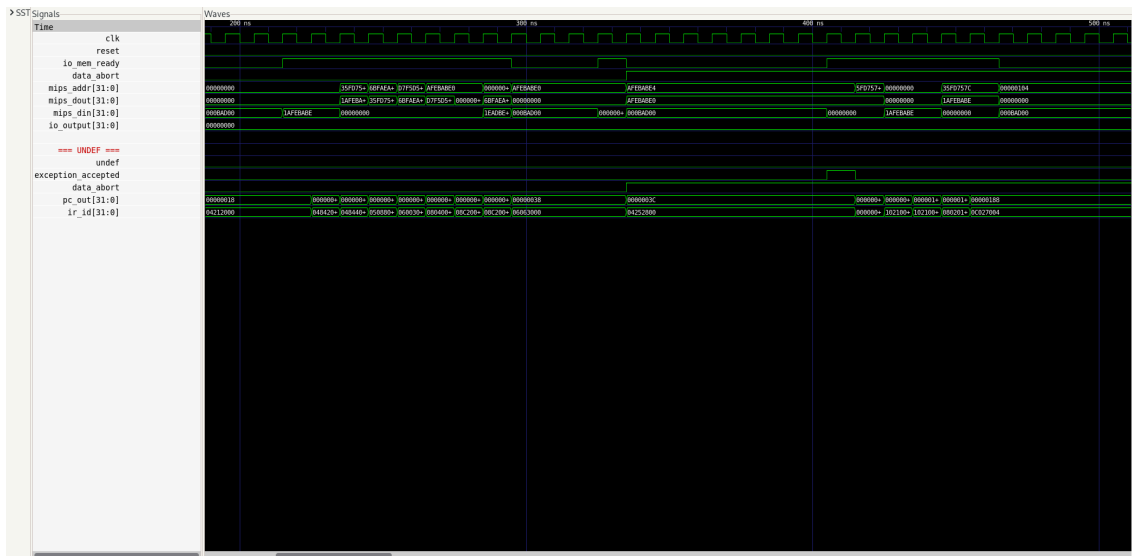
Snippet:

lw R1, 3(R0) ; acceso desalineado, debe activar Mem_ERROR

Test 9. UNDEF

Objetivo:

- demostrar entrada en la rutina de instrucción indefinida



Snippet:

```
.word 0xFFFFFFFF ; instruccion indefinida, fuerza salto al vector UNDEF
```

Tabla de cobertura por estado

Estado	Verificación principal
Inicio	tests 1..9
single_word_transfer_addr	tests 2, 4, 5, 7
Send_Addr	tests 2, 4, 5, 7
single_word_transfer_data	tests 2, 4, 5
block_transfer_addr	tests 1 y 3
read_block	tests 1 y 3
block_transfer_data	tests 1 y 3
Send_ADDR_CB	test 3
write_dirty_block	test 3
CopyBack	test 3
fallo	tests 7
bajar_Frame	tests 1 y 3

Análisis de speedup

El speedup se calcula frente a un sistema equivalente con solo MD, es decir, sin MC y sin aprovechamiento de localidad temporal. Para esta comparación se usa un modelo teórico, permitido por el guion, construido a partir de latencias medidas.

Caso de referencia:

- una primera lectura cacheable a una palabra de MD produce un read miss
- a continuación se repite la lectura de esa misma palabra N-1 veces

Coste en el sistema con MC:

- primer acceso: $A + CrB(MD) = 18.5$
- accesos posteriores: 0 ciclos de espera por ser read hit

Por tanto:

$$C_MC(N) = 18.5$$

Coste en el sistema equivalente con solo MD:

- cada lectura requiere $A + CrW(MD) = 7.5$

Por tanto:

$$C_MD(N) = 7.5 * N$$

El speedup queda:

$$\text{Speedup}(N) = C_MD(N) / C_MC(N) = (7.5 * N) / 18.5$$

Ejemplos:

Número total de lecturas N	C_MC(N)	C_MD(N)	Speedup
4	18.5	30	1.62
8	18.5	60	3.24
16	18.5	120	6.49

Conclusión:

- La jerarquía de memoria ofrece ventaja clara cuando existe reutilización de bloques ya cargados en caché.
- La mejora crece linealmente con el número de accesos repetidos tras el primer fallo.
- En patrones con poca reutilización o con reemplazos frecuentes, la ventaja se reduce y puede llegar a desaparecer.

Cuantificación de horas dedicadas

Miembro	Diseño UC	Integración	Validación	Memoria	Total
Polo Gracia, Gonzalo	11	7	6	4	28
Serrano Fernández, Alberto	13	8	8	7	36

Conclusiones y autoevaluación

Este proyecto ha consistido en el diseño e integración de una jerarquía de memoria basada en una memoria cache de dos vías para un procesador MIPS de 5 etapas. Entre los hitos alcanzados destacan la implementación de una Unidad de Control compleja, que mediante una máquina de estados es capaz de gestionar el protocolo copy-back, asegurando la coherencia de datos a la hora de reemplazar bloques sucios, además de diseñar con éxito la lógica de transferencia de bloques mediante una política de reemplazo FIFO, optimizando el aprovechamiento de la cache. Se ha integrado también el soporte para accesos a direcciones de memoria no cacheables, y se ha verificado la robustez del sistema frente a errores de bus y de alineación. El análisis de rendimiento ha permitido cuantificar la mejora significativa que aporta la cache frente a un sistema basado únicamente en memoria RAM.

El proyecto ha sido desafiante, sobre todo en lo que respecta a sincronización de señales de control con el bus o en la gestión del dirty bit y el control de los contadores, lo cual exigía precisión para mantener la coherencia de datos. A pesar de ello, se ha logrado un funcionamiento correcto y estable, comprendiendo cómo interactúan los distintos niveles de la jerarquía de memoria en un sistema real.